PDF-версия исходной лекции iSMART из сохранённого HTML. Содержание не заменено пересказом. Служебные элементы интерфейса исключены.

Классификация в Python - первые предсказания

Содержание

- Цель урока
- Планируемые результаты
- Ключевые понятия
- Проблемная ситуация
- Короткое объяснение темы
- Частые ошибки
- Итог урока
- Краткая памятка

Этот урок идёт после тем про обучающие и тестовые выборки, признаки, целевую переменную и типы задач машинного обучения. Обучающиеся уже понимают, что модель учится на примерах и получает на вход признаки объектов. Теперь они переходят к первому запуску классификации в Python.
Главный переход в этом уроке — от схемы машинного обучения к её программной реализации. Понятия «признак», «класс» и «обучающая выборка» будут представлены конкретными столбцами таблицы и переменными Python. Это помогает увидеть, как отдельные этапы работы с данными соединяются в один последовательный процесс.

Цель урока

Показать, как в Python запустить готовую модель классификации, передать ей признаки и правильные ответы, а затем получить предсказание для нового объекта.
Важно не только получить ответ от программы, но и понимать назначение каждого шага: почему целевой столбец отделяют от признаков, зачем оставляют тестовую часть данных и

почему новый объект должен быть описан теми же признаками, что и объекты обучающей выборки.

Планируемые результаты

- обучающийся объясняет, что такое классификация и чем признак отличается от целевой переменной;
- запускает готовый шаблон с библиотекой Scikit-learn и определяет назначение основных этапов программы;
- отделяет `X` и `y`, делит данные на обучающую и тестовую части;
- объясняет, почему тестовые объекты не должны участвовать в обучении модели;
- обучает простую модель дерева решений и получает предсказания для новых сообщений;
- проверяет, что новый объект содержит нужное количество признаков в правильном порядке;
- объясняет, какие признаки повлияли на итоговый класс.

Ключевые понятия

- классификация — задача, в которой модель выбирает один из заранее заданных классов; например, сообщение может относиться к классу обычных сообщений или к классу спама;
- объект — отдельный пример из набора данных, например одно сообщение;
- признаки — характеристики объекта, по которым модель учится; в таблице они обычно занимают отдельные столбцы;
- целевая переменная — правильный ответ, который модель должна предсказать; для каждого обучающего объекта этому ответу соответствует значение в столбце `label`;
- класс — одна из возможных категорий целевой переменной;
- обучающая выборка — данные, на которых модель находит закономерности между признаками и правильными ответами;
- тестовая выборка — новые для модели примеры, по которым можно посмотреть её работу после обучения;
- Scikit-learn — библиотека Python с готовыми инструментами машинного обучения;
- дерево решений — модель, которая идёт по цепочке вопросов и ветвлений, пока не приходит к итоговому классу;

- обучение модели — вызов метода `fit()`, во время которого модель ищет связи между признаками `X` и ответами `y`;
- предсказание — результат работы обученной модели для объекта, ответ которого заранее ей не передаётся.

> **Обрати внимание**
>
> Целевая переменная тоже хранится в таблице, но не является входным признаком. Во время обучения она играет роль правильного ответа, с которым модель связывает значения признаков.

Проблемная ситуация

Представим, что у нас есть много коротких сообщений. Часть из них обычные, часть похожа на спам, а некоторые находятся где-то посередине. Вручную проверять каждую строку долго. Значит, надо передать примеры модели и дать ей шанс научиться распознавать похожие случаи самостоятельно.
Сам текст сообщения модель из этого практикума напрямую не анализирует. Вместо него используются заранее подготовленные характеристики: длина сообщения, наличие ссылки, слова про скидку и срочного призыва. Так каждое сообщение превращается в строку числовых значений, с которой может работать алгоритм.
Например, у сообщения может быть большая длина, ссылка и слово про скидку, но не быть срочного призыва. Для модели это один объект с четырьмя признаками. Она сравнивает такое сочетание с примерами, показанными при обучении, и выбирает подходящий класс.
При этом один признак сам по себе не обязан однозначно определять ответ. Наличие ссылки встречается и в обычных сообщениях, поэтому дерево решений может учитывать сочетание нескольких характеристик.

Короткое объяснение темы

В этом практикуме берётся маленький табличный датасет сообщений. У каждого сообщения есть несколько признаков: длина, наличие ссылки, наличие слова про скидку и наличие срочного призыва. Отдельно хранится столбец `label` — правильный класс. Сначала таблица загружается в pandas. После загрузки полезно проверить названия столбцов, количество строк и первые записи: это позволяет убедиться, что данные прочитаны в ожидаемом виде.
Затем признаки отделяются от целевой переменной: `X` — это признаки, `y` — правильные ответы. Число строк в `X` и `y` должно совпадать, потому что каждой строке признаков

соответствует ровно один правильный ответ.
После этого данные делятся на обучающую и тестовую части. На обучающей части модель `DecisionTreeClassifier` проходит обучение. Тестовая часть временно остаётся в стороне: модель не использует её при вызове `fit()`. Благодаря этому предсказания можно получить для примеров, которые модель не видела во время обучения, а затем сравнить их с известными правильными ответами.

1. Загрузить таблицу и проверить её структуру.
2. Сформировать `X` только из столбцов признаков.
3. Сохранить столбец `label` в `y`.
4. Разделить `X` и `y` на согласованные обучающую и тестовую части.
5. Создать модель и обучить её методом `fit()`.
6. Передать тестовые или новые объекты в `predict()`.
7. Проверить полученные классы и сопоставить их с признаками объектов.

Дальше модели можно передать новый объект и получить предсказание. Новый объект описывается теми же четырьмя характеристиками. Если при обучении столбцы шли в порядке «длина, ссылка, скидка, срочность», значения нового сообщения должны быть расположены в том же порядке. Иначе программа может принять, например, значение признака ссылки за значение длины.
Метод `predict()` ожидает набор объектов, поэтому даже один новый объект передаётся как одна строка с несколькими признаками, а не как отдельная последовательность несвязанных значений. В результате метод возвращает предсказанный класс для каждой переданной строки.
Логика дерева решений удобна для первого знакомства: модель как будто задаёт несколько последовательных вопросов и по ответам движется к конечному классу. Например, один узел может проверять наличие ссылки, а следующий — значение другого признака. Конкретные вопросы и порядок ветвлений определяются по обучающим данным, поэтому их не следует задавать заранее без просмотра обученного дерева.

> **Важно**
>
> Обучение и предсказание — разные действия. `fit()` изменяет состояние модели, используя примеры и правильные ответы. `predict()` применяет уже найденные правила к переданным объектам и не заменяет этап обучения.

Маленький учебный датасет нужен для понимания последовательности действий, но по нескольким примерам нельзя делать вывод о надёжности классификатора в реальной задаче.

Если модель правильно классифицировала один новый объект, это показывает результат только для этого объекта, а не гарантирует правильную работу на любых сообщениях.

Мини-проверка

Определите роль каждого элемента: четыре характеристики сообщения, столбец `label`, метод `fit()` и метод `predict()`. Какие данные используются для обучения, а какие — для получения нового ответа?

Частые ошибки

- в `X` случайно оставили столбец `label`. Тогда правильный ответ попадает во входные данные, и модель получает подсказку, которой не будет у настоящего нового объекта;
- при создании нового примера нарушили порядок признаков. Даже если количество значений верное, каждое из них должно соответствовать тому же столбцу, что и при обучении;
- передали в `predict` не четыре признака, а меньше. Число признаков нового объекта должно совпадать с числом столбцов в обучающем наборе `X`;
- передали один объект не как строку набора данных, а как простой список отдельных признаков. Модель ожидает двумерную структуру: строки обозначают объекты, столбцы — признаки;
- перепутали обучающие и тестовые данные или обучили модель на всём наборе до проверки. В таком случае тестовая часть перестаёт быть новой для модели;
- вызвали `predict()` до `fit()`. Необученная модель ещё не построила правила классификации;
- решили, что один правильно предсказанный пример доказывает высокое качество модели. Для проверки нужно рассматривать несколько объектов тестовой выборки и сравнивать предсказания с правильными ответами;
- запустили ячейку, но не посмотрели на вывод и не заметили ошибку. Сообщение об ошибке обычно указывает на этап, где возникла проблема: загрузку данных, форму массива, число признаков или порядок выполнения ячеек.

Итог урока

После практикума обучающиеся получают первую рабочую связку: таблица признаков, простая модель Scikit-learn и предсказание для нового объекта. Этого уже хватает, чтобы перейти от

разговоров о классификации к настоящему запуску в Python.
Главный результат заключается не только в запуске готового шаблона, но и в понимании полного пути данных: исходная таблица превращается в `X` и `y`, обучающая часть используется в `fit()`, а тестовые и новые объекты передаются в `predict()`. На каждом этапе важно сохранять соответствие строк, столбцов и значений признаков.

Краткая памятка

- `X` — признаки, `y` — правильные ответы;
- каждой строке в `X` должен соответствовать один ответ в `y`;
- столбец `label` не должен оставаться среди входных признаков;
- обучающая часть нужна для поиска закономерностей, а тестовая — для проверки на не использованных при обучении примерах;
- `fit()` обучает модель;
- `predict()` даёт предсказание после обучения;
- новый объект должен содержать столько же признаков, сколько использовалось при обучении;
- порядок признаков для нового объекта должен совпадать с обучающей таблицей;
- одно предсказание показывает ответ для конкретного объекта, но не оценивает качество модели целиком;
- после каждой ячейки полезно проверять результат и сообщения об ошибках.

Материалы к занятию

Презентация содержит материалы для объяснения этапов классификации и сопровождения практического запуска модели.

- lesson_3_5_classification_python_updated.pptx (128.4 КБ)

Практическая часть

Содержание

- Материалы к практике

В этой части необходимо пройти основные этапы решения задачи классификации: загрузить размеченные данные, отделить признаки от правильных ответов, обучить модель и применить её к новым объектам. Такая последовательность важна: модель получает закономерности из обучающей таблицы, а затем использует их для выбора класса нового сообщения.
Перед началом скачайте ноутбук и CSV-файлы в одну папку. Это позволит обращаться к файлам по их именам без указания полного пути. Выполняйте ячейки ноутбука по порядку: результат более поздней ячейки может зависеть от переменных и библиотек, подготовленных ранее.

> **Важно**
>
> Признаки нового сообщения должны иметь те же названия, порядок и смысл, что и признаки обучающей выборки. Целевой столбец с правильным классом не включается в `X`: он используется отдельно как `y`.

Материалы к практике

Ноутбук содержит рабочую последовательность действий, а CSV-файлы — данные для обучения и отдельные примеры для предсказания. Сохраните все три файла перед запуском практической части.

- `lesson_3_5_classification_python.ipynb` (7.2 КБ) — ноутбук с ячейками для выполнения практики.
- `messages_new_examples.csv` (83 Б) — таблица новых примеров, для которых нужно получить прогнозы.
- `messages_train.csv` (550 Б) — обучающая таблица с признаками и известными ответами.

> **Если файл не найден**

> Проверьте точное имя файла, его расширение и папку, из которой запущен ноутбук. Имя в команде чтения должно совпадать с именем скачанного CSV-файла, включая символы подчёркивания.